



TEMA 06 • Trabalhando com Telas

Do Splash Screen à navegação entre telas: HTML, CSS e JavaScript aplicados à construção de interfaces interativas



Splash Screen



Manipulação do DOM



Temas Claro/Escuro



Formulários



Carrinho & Estoque



OneCompiler Ready



15 Questões com Gabarito



1. O que é uma "Tela" em um Aplicativo?



Conceito-chave: Uma **tela** é um conjunto de elementos visuais (HTML) organizados por estilos (CSS) que respondem a interações do usuário (JavaScript). Em aplicações web, cada "tela" geralmente é uma `<div>` com `id` próprio, que pode ser **exibida** ou **ocultada** conforme a necessidade.



Ciclo de vida de uma tela

Toda tela passa por 4 etapas fundamentais no desenvolvimento front-end:

1. ESTRUTURA

HTML

Define os elementos visíveis: títulos, campos, botões, imagens.

2. ESTILO

CSS

Aplica cores, espaçamentos, bordas e posicionamento.

3. COMPORTAMENTO

JavaScript

Reage a cliques, digitação e mudanças de valor.

4. TRANSIÇÃO

Troca de Tela

Alterna entre telas com `display: none/block` ou classes CSS.



2. Splash Screen – A Tela de Abertura

A **Splash Screen** é a primeira tela que aparece quando o aplicativo é aberto. Ela serve para:

- Mostrar a identidade visual (logo, nome do app).
- Exibir uma mensagem de carregamento ("Carregando...").
- Dar tempo para o sistema se preparar antes de mostrar o conteúdo principal.

 **Como funciona?** Usamos `setTimeout()` para agendar a troca de telas após alguns segundos. Cada `setTimeout` executa uma função depois de um tempo determinado (em milissegundos).

Estrutura de uma Splash Screen funcional

TELA 1

Splash (#splash)

Visível ao abrir. Mostra "Carregando..." e depois mensagens de progresso.

TELA 2

Conteúdo (#conteudo)

Começa com `display: none`. É exibida após 3 segundos.

TRANSIÇÃO

`setTimeout()`

Agenda a troca: splash desaparece, conteúdo aparece.

Detalhamento completo do Splash:

1. **0s:** A tela #splash aparece com "Carregando...".
2. **1s:** A mensagem muda para "  Preparando...".
3. **2s:** A mensagem muda para "  Quase pronto...".
4. **3s:** A #splash recebe `display: none` e a #conteudo recebe `display: block`.

3. Principais Conceitos Trabalhados nas Atividades

As 10 atividades a seguir exploram conceitos fundamentais que todo desenvolvedor front-end precisa dominar:

Atividade	Conceito Principal	Elemento HTML	Recurso JavaScript
1. Identificação	Captura de texto e exibição	<code><input type="text"></code>	<code>.value</code> , <code>.innerText</code>

Atividade	Conceito Principal	Elemento HTML	Recurso JavaScript
2. Carrinho	Cálculo com multiplicação	<code><input type="number"></code>	<code>parseFloat(), toFixed()</code>
3. Estoque	Comparação lógica	<code><input type="number"></code>	<code>if/else, .className</code>
4. Pagamento	Múltiplas decisões	<code><select></code>	<code>if/else if/else</code>
5. Endereços	Seleção exclusiva	<code><input type="radio"></code>	<code>querySelector()</code>
6. Cadastro	Múltiplos inputs	<code>text + number</code>	Concatenação de strings
7. Cadastro c/ Select	Select + Radio	<code><select>, radio</code>	<code>querySelector()</code>
8. Cadastro c/ Checkbox	Seleção múltipla	<code><input type="checkbox"></code>	<code>.checked</code>
9. Carrossel	Navegação entre imagens	<code></code>	<code>.src, variável de controle</code>
10. Tema Claro/Escuro	Troca de classes CSS	<code><body></code>	<code>.className</code>



4. Laboratórios Práticos (OneCompiler Ready)

Cada bloco abaixo é um arquivo completo. Clique em " **Abrir no OneCompiler**" para testar e modificar:



Lab 1: Splash Screen com Transição de Telas

Desafio de Teste: Abra no OneCompiler e observe as 3 mensagens de progresso aparecerem em sequência (1s, 2s, 3s) até a tela inicial substituir a splash.



lab1-splash-screen.html



Copiar



Abrir no OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
```

```
<title>Tela Splash</title>
<style>
  body { text-align: center; background: #f8fafc; font-family: sans-serif; }
  .card { background: white; max-width: 300px; margin: 30px auto; padding: 10px; }
  #splash { border: 3px solid #2563eb; }
  #conteudo { display: none; }
  h1 { color: #2563eb; }
</style>
</head>
<body>
  <div class="card" id="splash">
    <h1>📱 Meu Aplicativo</h1>
    <p id="mensagem">Carregando...</p>
```

📖 Entendendo o Lab 1 – Splash Screen

🏗️ Estrutura HTML

Temos **dois cards** no `<body>`:

- `#splash` — visível ao abrir. Contém o título do app e `<p id="mensagem">`.
- `#conteudo` — começa invisível (`display: none`). É a tela final.

🎨 CSS

O segredo está em `#conteudo { display: none; }`, que esconde a segunda tela até o momento certo. O `margin: 30px auto` centraliza o card horizontalmente (truque clássico do `auto`).

⚙️ JavaScript

Usamos **3 chamadas de** `setTimeout()`:

1. **1000ms (1s)**: altera o texto para "⚙️ Preparando..."
2. **2000ms (2s)**: altera para "🚀 Quase pronto..."
3. **3000ms (3s)**: esconde `#splash` e mostra `#conteudo`

Cada `setTimeout(função, tempo)` agenda a execução de uma função após `tempo` milissegundos.

🔄 Fluxo de Funcionamento

1. Página carrega → usuário vê `#splash` com "Carregando..."
2. Após 1s → "⚙️ Preparando..."
3. Após 2s → "🚀 Quase pronto..."

4. Após 3s → `#splash` desaparece, `#conteudo` aparece

🧠 Conceitos Importantes

Conceito	Onde aparece
Assincronismo	<code>setTimeout()</code>
Manipulação do DOM	<code>document.getElementById()</code>
Alteração de estilo	<code>.style.display = "block"</code>
Troca de telas	<code>display: none</code> ↔ <code>display: block</code>

🎵 Lab 2: Playlist Musical com Navegação

Desafio de Teste: Navegue pelas 3 músicas da playlist com os botões `←` e `→`. Observe como a variável `numero` controla qual música é exibida na tela.

lab2-playlist-musical.html

Copiar

Abrir no OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Playlist Musical</title>
  <style>
    body { text-align: center; background: #f5f3ff; font-family: sans-serif; }
    .card { background: white; max-width: 320px; margin: 30px auto; padding: 10px; }
    img { width: 200px; height: 200px; object-fit: cover; border-radius: 12px; }
    button { padding: 8px 15px; margin: 5px; background: #8b5cf6; color: white; border: none; }
    button:hover { background: #7c3aed; }
  </style>
</head>
<body>
  <div class="card">
    <h2>🎵 Minha Playlist</h2>
    
    <div id="titulo">Músicas 1</div>
```

📖 Entendendo o Lab 2 – Playlist Musical

Estrutura HTML

Um único `<div class="card">` com:

- `<h2>` — título fixo "🎵 Minha Playlist"
- `<img id="capa">` — capa que muda dinamicamente
- `<p id="titulo">` — nome da música
- Dois `<button>` com `onclick="anterior()"` e `onclick="proxima()"`

Os atributos `id="capa"` e `id="titulo"` são os "endereços" que o JavaScript usa para encontrar esses elementos.

CSS

Padrão de card centralizado:

- `max-width: 320px` — boa largura para mobile
- `margin: 30px auto` — o `auto` centraliza horizontalmente
- `border: 2px solid #8b5cf6` — borda roxa
- `object-fit: cover` — a imagem 200×200 preenche sem distorcer
- `button:hover` — feedback visual ao passar o mouse

JavaScript

Variável de estado: `let numero = 1;` guarda qual música está sendo exibida.

Função `mostrar()`: usa `if / else if / else` para decidir qual capa e título exibir, alterando `.src` (da imagem) e `.innerText` (do texto).

Função `proxima()`:

- `numero++` incrementa
- `if (numero > 3) numero = 1;` faz o ciclo circular
- Chama `mostrar()` para atualizar a tela

Função `anterior()`: lógica inversa com `numero--` e `if (numero < 1) numero = 3;`

Fluxo de Funcionamento

1. Página carrega → `numero = 1` (a capa inicial vem do HTML fixo)
2. Clique em  → `numero = 2` → `mostrar()` troca imagem/título
3. Após 3 → volta para 1 (carrossel circular)
4.  faz o mesmo no sentido inverso

🗨️ Conceitos Importantes

Conceito	Onde aparece
Manipulação do DOM	<code>document.getElementById()</code>
Eventos	<code>onclick="..."</code>
Estado da aplicação	variável <code>numero</code>
Condicionais	<code>if / else if / else</code>
Ciclo/carrossel	<code>numero > 3 → 1</code> e <code>numero < 1 → 3</code>
Separação HTML/CSS/JS	3 blocos independentes

🎨 Lab 3: Modo Claro e Escuro de uma Agenda

Desafio de Teste: Clique nos botões ☀️ Claro e 🌙 Escuro e observe a classe `.escuro` sendo aplicada e removida do `body`.

lab3-agenda-tema.html

Copiar

Abrir no OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Agenda de Contatos</title>
  <style>
    body { text-align: center; background: #ecfeff; color: #083344; font-fami
    .card { max-width: 300px; margin: 30px auto; padding: 20px; background: w
    h2 { color: #0891b2; }
    button { padding: 8px 15px; margin: 5px; border: none; border-radius: 8px
    .claro { background: #67e8f9; color: #083344; }
    .escuro { background: #164e63; color: white; }
    body.escuro { background: #083344; color: white; }
    body.escuro .card { background: #0e7490; border-color: #67e8f9; }
    body.escuro h2 { color: #67e8f9; }
  </style>
</head>
```

📖 Entendendo o Lab 3 – Agenda de Contatos

🏗️ Estrutura HTML

Um card único com:

- `<h2>` — título "📅 Agenda de Contatos"
- Dois botões: 🌞 `Claro` e 🌙 `Escuro`
- `<p id="saida">` — mostra o tema atual

Os botões usam `onclick="claro()"` e `onclick="escuro()"`.

🎨 CSS

A grande sacada está na classe `.escuro`:

- Definimos cores padrão (claro) no `body` e no `.card`
- Criamos `body.escuro { ... }` sobrescrevendo as cores
- Também estilizamos `body.escuro .card` e `body.escuro h2`

Isso é chamado de **tema baseado em classe**: uma única classe no `body` reestiliza tudo.

⚙️ JavaScript

Função `claro()`:

- `document.body.className = "";` — remove a classe "escuro"
- Atualiza o texto de saída para "🌞 Tema claro"

Função `escuro()`:

- `document.body.className = "escuro";` — aplica a classe
- Atualiza o texto para "🌙 Tema escuro"

🔄 Fluxo de Funcionamento

1. Página carrega → sem classe no `body` → tema claro padrão
2. Clique em 🌙 Escuro → classe `.escuro` adicionada → CSS troca as cores automaticamente
3. Clique em 🌞 Claro → classe removida → volta ao padrão

💡 Conceitos Importantes

Conceito	Onde aparece
Troca de classe	<code>document.body.className</code>
Seletores descendentes	<code>body.escuro .card</code>
Eventos	<code>onclick="claro()"</code>
Feedback textual	<code>.innerText</code>
CSS condicional	uma classe reestiliza vários elementos

Lab 4: Mochila de Viagem com Checkbox Múltiplo

Desafio de Teste: Marque vários itens da mochila e veja a string final concatenar todos eles separados por espaço.

 lab4-mochila-viagem.html

 Copiar

 Abrir no OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Mochila de Viagem</title>
  <style>
    body { text-align: center; background: #f0fdf4; font-family: sans-serif; }
    .card { background: white; max-width: 320px; margin: 30px auto; padding: 10px; }
    h2 { color: #15803d; }
    label { display: block; margin: 8px; text-align: left; }
    button { margin-top: 15px; padding: 10px 20px; background: #16a34a; color: white; }
    button:hover { background: #15803d; }
    #saida { padding: 10px; background: #dcfce7; border-radius: 8px; margin-top: 10px; }
  </style>
</head>
<body>
  <div class="card">
    <h2>🎒 Mochila de Viagem</h2>
```

Entendendo o Lab 4 – Mochila de Viagem

Estrutura HTML

Um card com:

- `<h2>` — título "  Mochila de Viagem"
- Três `<input type="checkbox">` com `id` único cada
- Um botão "Fechar Mochila" com `onclick="fecharMochila ()"`
- `<p id="saida">` — onde o resultado é exibido

Cada checkbox é envolvido por um `<label>`, o que torna o texto clicável também.

CSS

- `label { display: block; }` — cada checkbox em sua própria linha
- `#saida` tem fundo verde claro, mostrando o resultado final
- Botão com `hover` muda de tom (feedback visual)

JavaScript

Variável acumuladora: `let itens = "";` começa como string vazia.

Verificação de cada checkbox com `if (elemento.checked)`:

```
if (document.getElementById("roupas").checked) {  
    itens += "Roupas ";  
}
```

Se `.checked === true`, o nome do item é adicionado à string com `+=`.

Tratamento de caso vazio: se nenhum checkbox foi marcado, exibimos "Nenhum item selecionado".

Exibição: `document.getElementById("saida").innerText = "Mochila pronta:
" + itens;`

Fluxo de Funcionamento

1. Usuário marca os checkboxes que deseja
2. Clica em "Fechar Mochila"
3. JavaScript percorre os 3 `if`, acumulando os itens marcados
4. Mostra a string final no `#saida`

Conceitos Importantes

Conceito

Onde aparece

Propriedade `.checked`

verificação de cada checkbox

Concatenação de strings

`itens += "Roupas "`

Estrutura condicional

`if (...checked)`

Tratamento de caso vazio

`if (itens === "")`

Saída no DOM

`.innerText`

Lab 5: Navegação entre Múltiplas Telas (Splash → Cadastro → Confirmação)

Desafio de Teste: Este é o laboratório mais completo! Ele combina Splash Screen + Formulário + Troca de Telas em uma única aplicação.

 lab5-multiplas-telas.html Copiar Abrir no OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Múltiplas Telas</title>
  <style>
    body { text-align: center; background: #f8fafc; font-family: sans-serif; }
    .card { background: white; max-width: 320px; margin: 30px auto; padding: 10px; }
    #tela-splash { border: 3px solid #2563eb; }
    #tela-cadastro { display: none; }
    #tela-confirmacao { display: none; border: 3px solid #16a34a; }
    input { width: 90%; padding: 8px; margin: 6px 0; border-radius: 6px; border: 1px solid #ccc; }
    button { margin-top: 10px; padding: 10px 20px; background: #2563eb; color: white; }
  </style>
</head>
<body>
  <div class="card" id="tela-splash">
    <h1>  Meu App </h1>
  </div>
</body>
</html>
```

Entendendo o Lab 5 – Múltiplas Telas

Estrutura HTML

Este é o laboratório mais completo. Temos **três cards** no `<body>`:

- `#tela-splash` — visível ao abrir

- `#tela-cadastro` — começa escondida (`display: none`)
- `#tela-confirmacao` — também começa escondida

Cada tela tem seu próprio conjunto de elementos (título, inputs ou parágrafos).

CSS

- `#tela-cadastro { display: none; }` e `#tela-confirmacao { display: none; }` escondem as duas telas
- Bordas coloridas diferentes indicam visualmente cada tela (azul para splash, verde para confirmação)
- Inputs com `width: 90%` para ocupar quase toda a largura do card

JavaScript

Fase 1 – Splash: dois `setTimeout` agendam a transição:

- 1s: altera mensagem para "⚙️ Preparando..."
- 2s: esconde splash e mostra cadastro

Função `confirmar()`:

- Captura os valores de `#nome` e `#cidade`
- Monta mensagem personalizada
- Esconde `#tela-cadastro`, mostra `#tela-confirmacao`

Função `voltar()`: reverte — esconde confirmação, mostra cadastro novamente.

Fluxo de Funcionamento

1. Página carrega → `#tela-splash` visível
2. Após 1s → mensagem muda
3. Após 2s → splash some, cadastro aparece
4. Usuário preenche e clica em "Cadastrar" → confirmação aparece
5. Clique em "Voltar ao início" → retorna ao cadastro

Conceitos Importantes

Conceito	Onde aparece
Assincronismo	<code>setTimeout()</code>
Troca de telas	<code>.style.display = "none" / "block"</code>

Captura de inputs

`.value`

Concatenação de strings

`"Bem-vindo(a), " + nome`

Reutilização de telas

função `voltar()`

Fluxo multi-etapas

splash → cadastro → confirmação



5. Exercício de Fixação do Tema 06 (15 Questões)

Responda às 15 questões objetivas sobre telas, splash, manipulação do DOM e conceitos das atividades. Clique em **"Verificar Respostas"** ao final:

1 O que é uma "Splash Screen" em um aplicativo?

- ☒ A tela inicial de abertura que mostra o logo e uma mensagem de carregamento antes do conteúdo principal
- ☐ Um tipo de animação usada apenas em jogos
- ☐ O botão de desligar o aplicativo
- ☐ Uma tela de erro do sistema

2 Qual função JavaScript é usada para agendar a troca de telas na Splash Screen após um determinado tempo?

- ☒ `setTimeout()`
- ☐ `setInterval()`
- ☐ `delayTime()`
- ☐ `waitFor()`

3 Como ocultamos uma tela via CSS para que ela não apareça inicialmente?

☒ Usando `display: none;` no seletor da tela

☐ Usando `color: white;`

☐ Deletando o HTML da tela

☐ Removendo o CSS inteiro

4 Em uma galeria com navegação sequencial, `numero` ou `indiceAtual` ?
para que serve uma variável de controle como

☒ Armazenar qual item da sequência (imagem, música, filme) está sendo exibido no momento

☐ A cor de fundo da página

☐ O tamanho da fonte

☐ O número de cliques no botão

5 Para alternar entre tema claro e escuro, qual propriedade JavaScript alteramos no `<body>` ?

☒ `document.body.className`

☐ `document.body.innerHTML`

☐ `document.body.value`

☐ `document.body.src`

6 Qual método JavaScript é usado para capturar um elemento pelo seu `id` ?

☒ `document.getElementById("nomeId")`

☐ `document.querySelectorId("nomeId")`

☐ `document.getId("nomeId")`

☐ `document.findId("nomeId")`

7 Em um formulário com `input type="checkbox"`, qual propriedade JavaScript informa se a caixa está marcada?

☒ `.checked` — retorna `true` se marcado ou `false` se desmarcado

☐ `.value`

☐ `.innerText`

☐ `.className`

8 Em um formulário com botões de rádio (radio), como pegamos a opção selecionada?

☒ `document.querySelector('input[name="endereco"]:checked').value`

☐ `document.getElementById("endereco").value`

☐ `document.getRadio("endereco").value`

☐ `document.checked("endereco")`

9 Qual a função de `parseFloat()` em um carrinho de compras?

☒ Converter o valor de texto (String) do input para número decimal, permitindo cálculos matemáticos

☐ Arredondar o valor para cima

☐ Transformar número em texto

☐ Formatar o valor com vírgula

10 **Como exibimos um valor numérico com 2 casas decimais e vírgula no lugar do ponto?**

☒ `valor.toFixed(2).replace('.', ',')`

☐ `valor.format(2, ',')`

☐ `valor.toComma(2)`

☐ `valor.round(2).comma()`

11 **No controle de estoque, qual estrutura lógica usamos para verificar se a quantidade solicitada cabe no estoque?**

☒ Estrutura condicional `if / else`

☐ Laço de repetição `for`

☐ Função `setTimeout()`

☐ Comando `alert()`

12 **A Splash Screen permanece visível durante toda a execução do aplicativo, sem nunca ser ocultada.**

☐ ☒ Verdadeiro

☒ ☐ Falso

- 13 Um `name` em `input type="radio"` agrupa os botões para que apenas um seja selecionado por vez.

☒ Verdadeiro

☐ Falso

- 14 Para alternar entre telas, podemos usar `display: none/block`) manipulando o estilo dos elementos via JavaScript. Qual comando faz isso? CSS (

☒ `document.getElementById("tela").style.display = "block";`

☐ `document.getElementById("tela").display = true;`

☐ `document.trocarTela("tela");`

☐ `document.getElementById("tela").show();`

- 15 Em uma aplicação bem estruturada, qual é a divisão de responsabilidades entre HTML, CSS e JavaScript?

☒ HTML = estrutura; CSS = estilo; JavaScript = comportamento e lógica

☐ HTML = estilo; CSS = lógica; JavaScript = estrutura

☐ HTML = banco de dados; CSS = servidor; JavaScript = rede

☐ Os três fazem exatamente a mesma coisa

☒ Verificar Respostas

☐ Limpar Respostas

 Relatório de Desempenho do Aluno

15 / 15

✓ **Questão 1: Correta!**

A Splash Screen é a tela de abertura que exibe o logo e uma mensagem de carregamento antes do conteúdo principal.

✓ **Questão 2: Correta!**

setTimeout() agenda a execução de uma função após um tempo especificado em milissegundos.

✓ **Questão 3: Correta!**

display: none; oculta o elemento — essencial para telas que começam invisíveis (como #conteudo, #cadastro) e são reveladas via JavaScript.

✓ **Questão 4: Correta!**

A variável de controle (ex: numero, indiceAtual) guarda a posição atual na sequência. É incrementada ou decrementada pelos botões de navegação.

✓ **Questão 5: Correta!**

document.body.className é a propriedade que define a(s) classe(s) CSS do body.

✓ **Questão 6: Correta!**

getElementById() é o método padrão para selecionar um elemento pelo seu id.

✓ **Questão 7: Correta!**

A propriedade .checked é exclusiva de checkboxes e radios, retornando true/false conforme o estado do campo.

✓ **Questão 8: Correta!**

querySelector com :checked seleciona o radio que está marcado no grupo.

✓ **Questão 9: Correta!**

parseFloat() converte a string do input em número decimal para permitir cálculos.

✓ **Questão 10: Correta!**

toFixed(2) limita a 2 casas decimais; replace(".", ",") troca o ponto pela vírgula.

✓ **Questão 11: Correta!**

A estrutura if/else é ideal para comparar a quantidade solicitada com o estoque disponível.

✓ **Questão 12: Correta!**

Falso. A Splash Screen é exibida apenas no início e depois é ocultada para dar lugar ao conteúdo.

✓ **Questão 13: Correta!**

Verdadeiro. Radios com o mesmo name formam um grupo onde apenas um pode estar marcado.

✓ **Questão 14: Correta!**

style.display = "block" torna o elemento visível novamente após ter sido ocultado.

✓ **Questão 15: Correta!**

HTML = estrutura; CSS = aparência; JavaScript = comportamento e lógica das decisões.



Imprimir / Salvar em PDF

📌 **ATENÇÃO ALUNOS:** Após testar os 5 laboratórios no OneCompiler e concluir o Quiz, salve os seus códigos em formato HTML e envie o relatório pelo formulário oficial.

**Formulário Oficial de Entrega****Formulário de Entrega do Relatório**

Envio dos arquivos HTML + Atividades de Telas



Material Pedagógico: Tema 06 • Disciplina: Programação Mobile • 2026